SCSM2223 CROSS-PLATFORM APPLICATION DEVELOPMENT

MOCK FINAL EXAMINATION (SET C) — SEMESTER 2, 2025/2026

ANSWER SCHEME AND EXAMINER NOTES

Hardest of the three sets: code tracing, error hunting and step ordering. Sit it under full exam conditions.

SECTION A — MULTIPLE-CHOICE KEY (20 × 1 MARK)

No.	Ans	Why
1	**C**	Stateless: every request independent; no server memory of previous ones.
2	**A**	Viewport meta = correct scaling on mobile; without it pages render zoomed out.
3	**D**	One external file styles many pages + browser caching.
4	**B**	textContent = plain text (safe); innerHTML parses markup.
5	**A**	filter() → NEW array of passing elements (find() returns the first one).
6	**C**	JSON.parse: string → JS object.
7	**B**	$("#msg") selects by ID; .text() sets the text content.
8	**D**	1, 3, 2 — the setTimeout callback waits in the queue until the stack is empty.
9	**A**	await only directly inside an async function.
10	**C**	.number casts the bound value to a Number.
11	**B**	Only "Slim".toLowerCase() contains "s". ("pdo" has no s.)
12	**D**	Slot = parent injects HTML into the child's placeholder.
13	**A**	Lazy loading: code downloaded on first visit → smaller initial bundle.
14	**C**	err.request without err.response = sent but nothing came back (server down).
15	**B**	Duplicate UNIQUE value → 409 Conflict (lab: ER_DUP_ENTRY).
16	**D**	Onion model: last added = outermost = first to run, so it wraps all layers.
17	**A**	Query string → getQueryParams(). Body → getParsedBody(). Path → $args.
18	**B**	Atomicity: rollBack() undoes everything since beginTransaction().
19	**C**	Expired exp → ExpiredException → middleware answers 401.
20	**D**	The * origin cannot be combined with credentials:true (browser blocks it).

⚠ ***Trap to watch:*** *Q8 and Q11 are code-reading items — slow down and execute the code mentally, line by line. Q16 is the onion rule the Ch9 slides state verbatim: error middleware added LAST so it runs FIRST.*

SECTION B — MODEL ANSWERS

QUESTION 1 (8 marks) — Architecture and HTTP

a) Correct order: B → E → F → C → A → D. (Browser sends request → web server hits public/index.php → Slim parses into PSR-7 object → middleware stack runs → router dispatches to handler → handler builds the response.)

4 marks: all six correct = 4; one element misplaced = 3; two misplaced = 2; first and last correct only = 1.

b) The client itself carries the proof of identity: after login the server issues a signed JWT and the client re-sends it in the Authorization: Bearer header on every request [1]. The server verifies the SIGNATURE (and expiry) with its secret key — no session lookup, nothing stored server-side, so each request is self-contained, which matches the stateless constraint [1].

c) Safe = the request does not change server state (read-only) [0.5]. Idempotent = repeating the request gives the same result as one call [0.5]. GET is BOTH safe and idempotent [1].

QUESTION 2 (10 marks) — Asynchronous JavaScript

a) Order: start, end, timer, data [2]. Justification: “start” and “end” are synchronous statements executed immediately on the call stack; the setTimeout callback — even with a 0 ms delay — is placed in the callback queue and the event loop only moves it onto the stack AFTER the stack is empty, so “timer” must print after “end” [2]. (“data” is last because the network response takes ~200 ms.)

⚠ ***Trap to watch:*** *A 0 ms timeout never runs immediately — this is THE event-loop question. If you wrote start, timer, end — re-read the Ch5 event loop material tonight.*

b) Model answer:

```
async function createBook() {
  try {
    const response = await fetch("http://localhost:8000/api/books", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title: "Clean Code", author: "Martin" })
    });
    if (!response.ok) {
      throw new Error(`HTTP Error! Status: ${response.status}`);
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error:", error.message);
  }
}
```

Marks: method: "POST" in the options object [1]; Content-Type header [1]; body: JSON.stringify(...) [1]; ok-check + await response.json() [1]. (try/catch shown is good practice; not penalised if omitted since the question didn’t demand it.)

⚠ ***Trap to watch:*** *body must be a STRING — body: { title: ... } without JSON.stringify is the classic fetch-POST error. Headers spelling matters: "Content-Type".*

c) .catch() handles a rejected Promise anywhere earlier in the chain [1]; .finally() runs in every outcome — typically used to stop a loading spinner [1].

QUESTION 3 (12 marks) — Vue Reading and Reasoning

a) i) `0 / 0` [1]. **ii)** `2 / 4` [1]. **iii)** Console shows exactly two lines, in order: `count: 0 -> 1` then `count: 1 -> 2` [2].

⚠ ***Trap to watch:*** *The computed double updates in the DOM but logs nothing — only the watcher writes to the console. Watch callbacks receive (newValue, oldValue) in THAT order; the template string here prints old -> new.*

b) A <slot> is a placeholder inside a child's template where the PARENT injects its own HTML [1]. A named slot is declared as <slot name="title"> and the parent fills it with <template #title>...</template> [1]. If the parent provides nothing, the slot renders its fallback (default) content written between the <slot> tags [1].

c) i) State = `items` (ref); getter = `itemCount` (computed); action = `add()` (function) [1.5]. **ii)** Model answer [1.5]:

```
const store = useCartStore()
store.add(p)
```

d) .trim strips leading/trailing whitespace from the bound value [1]; .lazy syncs on the change event (when the field loses focus / Enter) instead of on every keystroke [1].

QUESTION 4 (15 marks) — Security and Deployment Scenario

a) Session-based: the server stores a session record and the browser holds only a session ID cookie — every request triggers a server-side lookup; the server is stateful [1]. Token-based (JWT): the server stores nothing; the client holds a signed token and the server merely verifies the signature each request [1]. The course uses JWT because a REST API must be stateless — tokens make every request self-contained and scale across servers without shared session storage [1].

b) i) IDOR — Insecure Direct Object Reference (broken access control) [1]. **ii)** The JWT middleware decodes the token and attaches the verified payload to the request with withAttribute('auth', ...) — the route reads that verified identity, never trusting client input [1.5]. **iii)** 404: the resource does not exist at all. 403: it exists, the caller is authenticated, but is neither the owner (created_by ≠ sub) nor an admin — authorization fails [1.5].

c) Defense in depth = multiple overlapping controls, so if any one layer fails the others still stop the attacker [1]. Layers [3]: Network — HTTPS/firewall/no public DB ports; Edge — WAF/rate limiting/bot protection; App — validation, escaping, prepared SQL, JWT, role checks; Data — hashed passwords, encrypted secrets, least-privileged DB user; Operations — logging, monitoring, alerts, updates.

d) Correct order: B → D → A → E → C [2]. D before A because npm run build compiles the Vue project into dist/ with the env variable INLINED into the bundle — npx cap sync merely COPIES dist/ into the native Android project. Syncing before building (or building with the wrong URL) ships a stale or localhost-pointing bundle into the APK [2].

SECTION C — MODEL CODE ANSWERS

QUESTION 5 (10 marks) — DOM Debugging, jQuery, CSS

a) The three errors [1 each]: 1. `getElementByld` is misspelled — it must be `getElementById` (lowercase L vs capital I trick). 2. `addEventListener(click, …)` — the event name must be the STRING `"click"`. 3. `querySelector(".taskList")` selects a CLASS, but taskList is an ID — must be `"#taskList"` (or getElementById). Corrected program [1]:

```
const btn = document.getElementById("addBtn");

btn.addEventListener("click", function () {
  const input = document.getElementById("task");
  const item = document.createElement("li");
  item.textContent = input.value;
  document.querySelector("#taskList").appendChild(item);
  input.value = "";
});
```

⚠ ***Trap to watch:*** *The lowercase-l/capital-I swap is invisible in most fonts — real papers love it. Always sanity-check: # for id, . for class.*

b) Model answer:

```
$(document).ready(function () {
  $("#loadBtn").click(function () {
    $.get("data.json", function (data) {
      $("#out").text(data.message);
    });
  });
});
```

Marks: $(document).ready wrapper [1]; click handler on #loadBtn [1]; $.get with callback writing data.message via .text() [1].

c) Model answer:

```
:root {
  --brand: #42B883;
}

.btn {
  background-color: var(--brand);
}

.btn:hover {
  background-color: #2f8e63;
}
```

Marks: :root custom property [1]; var(--brand) usage [1]; :hover state [1].

QUESTION 6 (10 marks) — BookSearch.vue

Model answer:

```
<template>
```

```
  <div>
    <input v-model="searchText" placeholder="Search books">
    <p v-if="filteredBooks.length === 0">No books found.</p>
    <ul>
      <li v-for="book in filteredBooks" :key="book.id">
        {{ book.title }}
      </li>
    </ul>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue'

const props = defineProps({
  books: { type: Array, required: true }
})

const searchText = ref('')

const filteredBooks = computed(() =>
  props.books.filter(b =>
    b.title.toLowerCase().includes(searchText.value.toLowerCase())
  )
)
</script>
```

Marks: SFC + script setup structure [1]; defineProps Array required [2]; ref searchText + v-model [1]; computed with case-insensitive filter (both sides toLowerCase) [3]; v-for over filteredBooks (NOT books) with :key="book.id" [2]; v-if empty state [1].

⚠ ***Trap to watch:*** *Two killers here: (1) inside the computed you need props.books (the prop must be captured from defineProps) and searchText.value; (2) looping over books instead of filteredBooks silently ignores the search — read what the v-for iterates.*

QUESTION 7 (15 marks) — Slim Group, PDO Transaction, JWT Middleware

a) Model answer:

```
$app->group('/api', function ($group) {
   $group->delete('/books/{id}', function (Request $req, Response $res, array
$args) {
      $id = (int)$args['id'];

      if (!deleteBook($id)) {
         $res->getBody()->write(json_encode(['error' => 'Book not found']));
         return $res
            ->withHeader('Content-Type', 'application/json')
            ->withStatus(404);
      }

      $res->getBody()->write(json_encode(['message' => 'Book deleted']));
      return $res
         ->withHeader('Content-Type', 'application/json')
         ->withStatus(200);
   });
});
```

Marks: group('/api', ...) wrapper with the route INSIDE it (path therefore '/books/{id}', not '/api/books/{id}') [1]; delete verb + {id} + $args cast [1]; failure branch 404 with error JSON [1]; success 200 with message [1]; header + returns in both branches [1].

⚠ ***Trap to watch:*** *Inside a group the route path must NOT repeat the prefix — writing $group->delete('/api/books/{id}') doubles the prefix and would 404. Also note the course convention returns 200 with a body on delete (the slides mention 204 as an alternative — either accepted if consistent).*

b) Model answer:

```
try {
    $pdo->beginTransaction();

    $stmt = $pdo->prepare('UPDATE books SET stock = stock - 1 WHERE id
= :id');
    $stmt->execute([':id' => $bookId]);

    $stmt = $pdo->prepare('INSERT INTO orders (book_id, user_id) VALUES
(:bid, :uid)');
    $stmt->execute([':bid' => $bookId, ':uid' => $userId]);

    $pdo->commit();
} catch (PDOException $e) {
    $pdo->rollBack();
    throw $e;
}
```

Marks: beginTransaction before the statements [1]; both statements as prepare + execute with placeholders [2]; commit on success [1]; catch PDOException → rollBack + re-throw [1].

⚠ ***Trap to watch:*** *Order matters: begin BEFORE the first statement, commit as the LAST line of the try. rollBack() belongs in catch — never after commit. This is the ACID/atomicity question in code form.*

c) Model answer:

```
$header = $req->getHeaderLine('Authorization');
$token = str_replace('Bearer ', '', $header);

try {
    $decoded = JWT::decode($token, new Key($secret, 'HS256'));
} catch (Exception $e) {
    return jsonResponse($res, ['error' => 'Invalid or expired token'], 401);
}
```

Marks: getHeaderLine('Authorization') [1]; stripping the 'Bearer ' prefix [0.5]; JWT::decode with new Key($secret, 'HS256') inside try [1]; catch → 401 JSON [0.5].

d) Rate limiting on /auth/login [1]; when the limit is exceeded the server returns 429 (Too Many Requests) with the Retry-After header telling the client when it may try again [1].

THE COMPLETE THREE-SET TRAINING SYSTEM

Difficulty ladder: Set A = write from a clear specification (baseline). Set B = diagnose and repair (spot-and-fix, rewrite bad designs). Set C = trace, order and integrate (predict output, order lifecycles and pipelines, read unfamiliar code, transactions and middleware). The real paper will mix all three modes — after these sets, no question FORMAT can surprise you.

Blueprint constant across all sets: 100 marks / 3 hours; A 20 MCQ (Remember ≈ 20%), B 45 structured (Understand ≈ 45%), C 35 code (Apply ≈ 35%); backend stack always the largest block (~33–35 marks), Vue second (~26), async third (~13) — exactly the Dr's published mark distribution and topic priorities.

Final-week protocol: Day 1: sit Set C cold, mark it, list every lost mark. Day 2: retype every imperfect code answer until it runs (Slim route, transaction, fetch POST, BookSearch component). Day 3: re-sit your weakest earlier set (A or B) — the score jump tells you the revision worked. Night before 7 July: read the three MCQ key tables and the trap notes only. Sleep. The tips document says all-nighters cost more marks than they add — believe it.